

Lab

Exceptions

Lab: Exceptions

Objective

In this lab, you will:

- Develop the ability to throw and catch exceptions effectively.
- Gain a comprehensive understanding of how the Java runtime propagates exceptions through the call stack until an appropriate handler is located.

Overview

First, you will address exceptions by implementing logic in the Account class to throw an exception object whenever the balance goes into the red.

In the second part of this lab, you will enhance the Motorway Simulator by introducing functionality to throw an exception when the Registration Factory runs out of available registration plates.

Finally, in the third part of this lab, you will work on managing Java's checked exceptions and learn how to handle them effectively.

Part 1

Using try/throw/catch in the Account class.

1. Add a new package to the **Labs** project called **lab6**.
2. Create a class in this package called **Program** with a *main()* method in the lab6 package.
3. Create a class called **Account** with the following fields and methods:

int id;

double balance;

String owner;

- a) Create a constructor to set these fields.
- b) Add the following method:

void withdraw(double amount) *// reduces the balance by amount*

void deposit(double amount) *// increases the balance by amount*

void close()

Just display a message like 'account 123 is closed' where 123 is the **ID**.

String getDetails() to get details (id, balance and owner).

Add suitable code to the withdraw, deposit, and getDetails() methods.

4. In the main() method, create an account with an initial balance of £100.
5. Perform a withdrawal of **£50**, then use the **getDetails()** method to display the account's information.
6. Attempt to withdraw **£60** and display the account's details afterwards.
(Note: Your friendly bank currently has no withdrawal limits!)
7. Modify the withdraw() method to throw an **IllegalArgumentException** if the balance becomes negative, rather than allowing an overdraft.
8. In the main() method, catch this exception and provide a user-friendly error message explaining the issue.
9. Build and run the program, ensuring that the Account class successfully throws the exception and that it is properly handled in the main() method.
10. Enhance the exception handler in main() by adding a finally block to close the account, regardless of whether an exception occurs or not.

Part 2 — (Optional, if you've completed the challenge exercise)

1. Let's get back in the motorway simulator lab which you created before.
2. The factory class will at some time run out of registration plates.
How would you handle this situation?
How would you inform the Vehicle class that we've ran out of reg plates?
How would the Vehicle class inform the caller we cannot create a vehicle?
Implement the required exception mechanisms the best you can.

Part 3

1. Create a method in the Program class to write a message to a log file.
2. Type the following code:

```
private static void log(String msg) throws IOException {  
    File file = new File("log45.txt");  
    FileReader fr = new FileReader(file);  
    int c = fr.read();    // try to read a single  
    fr.close();  
}
```

3. Call the log method from main().
4. Your code will not compile unless you change the signature of the log() method. It should indicate that it throws **IOException**.
5. Catch this exception in main() and display a **IOException**'s message, should the log() method fail.



QA.com